

Tri-Blaster

Shooter arcade 2D en Unity

Eva Gallardo Romero

2 de marzo de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 4

5 Documentación técnica **4**
5.1 Director.cs y AbilityHolder.cs 4
5.2 Habilidades/Habilidad1.cs a Habilidad4.cs 5
5.3 scripts/Player.cs 6
5.4 scripts/Spawner.cs, Enemy.cs y HealthEnemy.cs 7
5.5 scripts/HealthPlayer.cs, GameEvents.cs y Score.cs 7

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Tri-Blaster.

El juego se puede probar en el navegador en itch.io y la documentación del código generada con Doxygen está en evagr11.github.io/Tri-Blaster.

1 Descripción general

Tri-Blaster es un **shooter arcade en 2D** hecho en **Unity** con **C#** en la asignatura de Programación de 1.º de DAM. El jugador es un **triángulo** en el centro de la pantalla que gira siempre hacia el **ratón**. Desde fuera de la pantalla llegan **círculos enemigos** (uno por segundo) que van directos hacia él; si le tocan, le quitan vida.

Para defenderse tiene **cinco habilidades**, cada una con su tiempo de recarga. Se elige una con las teclas **1 a 5** y se usa con el **clic izquierdo**. Los iconos de la parte de abajo se van rellenando mientras se recargan.

Tecl	Habilidad	Recarga	Efecto
1	Disparo 1	0,5 s	Proyectil pequeño hacia donde apunta el triángulo.
2	Disparo 2	2 s	Proyectil más grande (escala 1,25 en lugar de 0,75).
3	Curación instantánea	5 s	Recupera toda la vida.
4	Curación	3 s	Multiplica por 5 la regeneración de vida durante 10 segundos.
5	Aumento de fuerza	2 s	Duplica la fuerza durante 6 segundos: proyectiles más grandes y con más daño.

La vida del jugador y la de cada enemigo se ven con un **degradado de color**. La partida empieza con **Espacio**; al perder aparece una ventana con los puntos conseguidos y se reinicia con **R**. Cada enemigo eliminado suma un punto.

2 Diagrama de clases

Las habilidades heredan de la clase abstracta `Director`, que obliga a tener un método `Trigger`. `AbilityHolder` guarda las cinco y lanza la elegida. Los avisos entre objetos (muerte del jugador, muerte de un enemigo e inicio de partida) van con **eventos** estáticos de `GameEvents`.

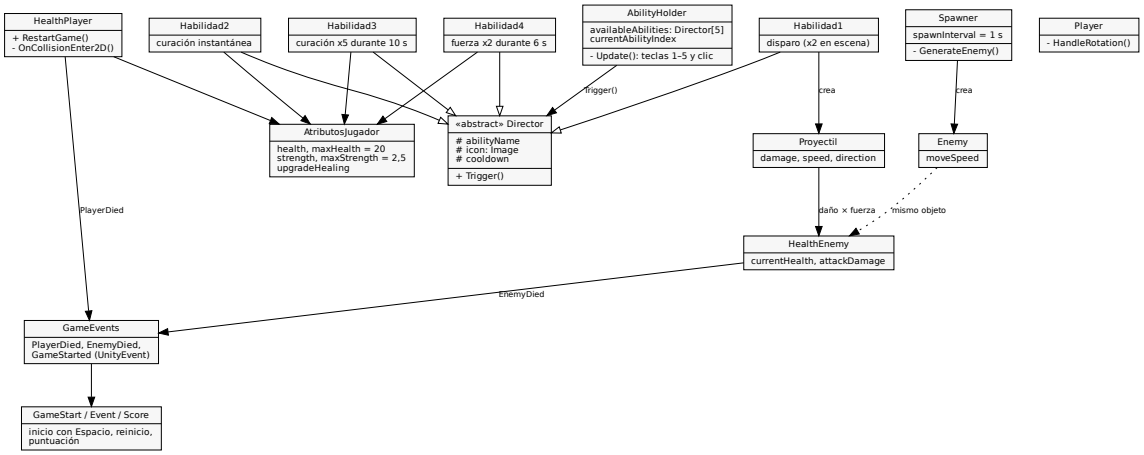
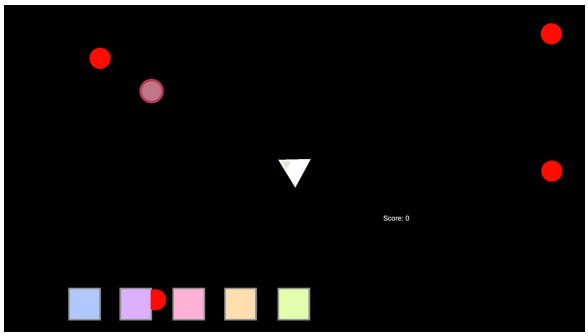


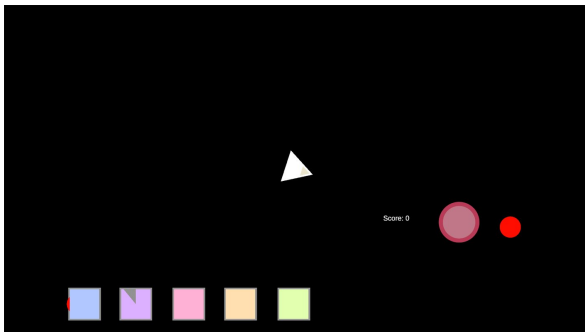
Figura 1: Diagrama de clases de Tri-Blaster

3 Diseño

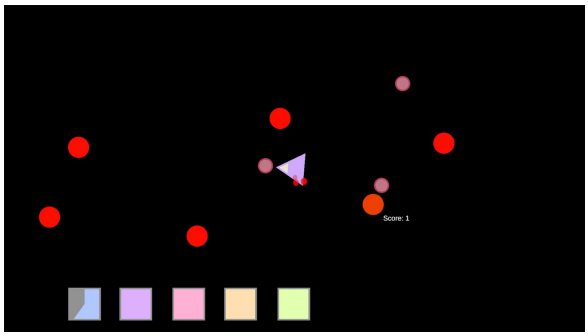
Fondo negro, el jugador como triángulo blanco que cambia de color según su vida, enemigos como círculos que pasan del rosa al rojo, la puntuación a la derecha y los cinco iconos de habilidad en colores pastel abajo a la izquierda.



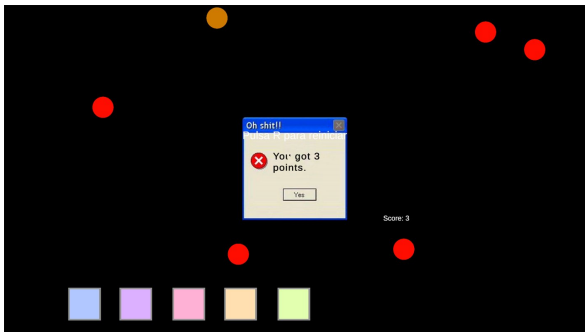
Disparo hacia el ratón



Disparo grande recargándose



Partida en marcha



Ventana de derrota

Figura 2: Capturas de una partida

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La partida debe empezar al pulsar **Espacio** y reiniciarse con **R**.
RF02	El triángulo debe **girar hacia el ratón** durante la partida.
RF03	Debe aparecer un enemigo **cada segundo** en un punto al azar de un círculo alrededor del jugador, y avanzar hacia él.
RF04	Al chocar, el enemigo debe quitar vida al jugador y desaparecer.
RF05	Las teclas **1–5** deben elegir la habilidad y el **clic izquierdo** usarla si está recargada.
RF06	El daño de los proyectiles debe multiplicarse por la **fuerza** del jugador, y su tamaño crecer con ella.
RF07	El jugador debe **regenerar vida** poco a poco, sin pasar de la vida máxima.
RF08	La vida del jugador y de los enemigos debe verse con un **degradado de color**.
RF09	Cada enemigo eliminado debe sumar un **punto**, y al perder se deben mostrar los puntos.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El juego debe estar hecho en **Unity** con scripts en **C#**.
RNF02	Todas las habilidades deben heredar de una clase abstracta común para poder añadir más sin tocar `AbilityHolder`.
RNF03	Los objetos se comunican con **UnityEvent** para no depender unos de otros.
RNF04	Los tiempos de recarga y los efectos se configuran desde el Inspector.
RNF05	Debe poder jugarse en el navegador (versión WebGL en itch.io).
RNF06	Los scripts deben tener comentarios de documentación `///` para Doxygen.

5 Documentación técnica

Este anexo agrupa la documentación por scripts, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `using` y los comentarios `///`.

5.1 Director.cs y AbilityHolder.cs

5.1.1 Ubicación

`Assets`

Descripción general:
`Director` es la clase base de las habilidades y `AbilityHolder`, la que las guarda y las lanza.

5.1.2 Funciones

Director **Descripción:** Datos comunes a todas las habilidades y el método que cada una implementa.

Código comentado:

```
1 public abstract class Director : MonoBehaviour
2 {
3     [SerializeField] protected string abilityName;
4     [SerializeField] protected Image icon;        // se rellena al recargar
```

```
    [SerializeField] protected float cooldown;
    public abstract void Trigger();
}
```

AbilityHolder.Update **Descripción:** El clic lanza la habilidad elegida y las teclas 1–5 cambian de habilidad, sin salirse del array.

Código comentado:

```
private void Update()
{
    if (availableAbilities.Length > 0 && currentAbilityIndex >= 0 &&
        currentAbilityIndex < availableAbilities.Length)
    {
        if (habilitiesActive && Input.GetMouseButtonDown(0))
            availableAbilities[currentAbilityIndex].Trigger();
    }
    if (Input.GetKeyDown(KeyCode.Alpha1)) { currentAbilityIndex = 0; }
    if (Input.GetKeyDown(KeyCode.Alpha2)) { currentAbilityIndex = 1; }
    if (Input.GetKeyDown(KeyCode.Alpha3)) { currentAbilityIndex = 2; }
    if (Input.GetKeyDown(KeyCode.Alpha4)) { currentAbilityIndex = 3; }
    if (Input.GetKeyDown(KeyCode.Alpha5)) { currentAbilityIndex = 4; }

    if (currentAbilityIndex < 0)
        currentAbilityIndex = 0;
    else if (currentAbilityIndex >= availableAbilities.Length)
        currentAbilityIndex = availableAbilities.Length - 1;
}
```

5.2 Habilidades/Habilidad1.cs a Habilidad4.cs

5.2.1 Ubicación

`Assets/Habilidades`

Descripción general:
Las habilidades. Todas descuentan su recarga en `FixedUpdate` y rellenan el icono; se muestra la recarga una vez y el `Trigger` de cada una.

5.2.2 Funciones

Recarga **Descripción:** Común a todas: el icono se rellena de 0 a 1 mientras baja el tiempo.

Código comentado:

```
private void FixedUpdate()
{
    if (currentCooldown > 0f)
    {
        currentCooldown -= Time.deltaTime;
        icon.fillAmount = 1 - (currentCooldown / cooldown);
    }
}
```

Habilidad1.Trigger **Descripción:** Crea un proyectil en la punta del triángulo, en la dirección a la que mira y con un tamaño proporcional a la fuerza.

Código comentado:

```
if (gameStartScript.gameStarted && currentCooldown <= 0)
{
    player = GameObject.FindGameObjectWithTag("Player");
    spawnedBullet = Instantiate(projectileTemplate, projectileSpawnPoint.position,
        Quaternion.identity);
    bulletBehavior = spawnedBullet.GetComponent<Proyectil>();
    playerStats = player.GetComponent<AtributosJugador>();

    bulletBehavior.direction = player.transform.up;          // hacia donde apunta
```

```
9       spawnedBullet.transform.localScale = Vector3.one * (scaleBase * playerStats.
            strength);
10
11      currentCooldown = cooldown;
12      icon.fillAmount = 0;
13  }
```

Habilidad2.Trigger **Descripción:** Cura de golpe sumando la vida máxima (`HealthPlayer` la limita después) y enseña partículas durante un segundo.

Código comentado:

```
1  if (gameStartScript.gameStarted && currentCooldown <= 0)
2  {
3      healingEffect.Play();
4      playerStats = GameObject.FindGameObjectWithTag("Player").GetComponent<
           AtributosJugador>();
5      playerStats.health += playerStats.maxHealth;
6      Invoke("StopHealingEffect", 1f);
7      currentCooldown = cooldown;
8      icon.fillAmount = 0;
9  }
```

Habilidad3 y Habilidad4 **Descripción:** Multiplican la regeneración (x5, 10 s) o la fuerza (x2, 6 s) y deshacen el cambio con `Invoke`.

Código comentado:

```
1  // Habilidad3
2  playerStats.upgradeHealing *= healingMultiplier;      // 5 en la escena
3  Invoke("StopHealingBoost", 10);
4
5  private void StopHealingBoost()
6  {
7      playerStats.upgradeHealing /= healingMultiplier;
8  }
9
10 // Habilidad4
11 playerStats.strength *= strengthFactor;               // 2
12 Invoke("ResetStrengthBoost", 6f);
13
14 private void ResetStrengthBoost()
15 {
16     playerStats.strength /= strengthFactor;
17 }
```

5.3 scripts/Player.cs

5.3.1 Ubicación

`Assets/scripts/Player.cs`

Descripción general:
Gira el triángulo hacia el ratón.

5.3.2 Funciones

HandleRotation **Descripción:** Pasa la posición del ratón a coordenadas del mundo y calcula el ángulo respecto al eje vertical.

Código comentado:

```
1  private void HandleRotation()
2  {
3      Vector3 mousePosition = Camera.main.ScreenToWorldPoint(Input.mousePosition);
4      mousePosition.z = 0;
5      Vector3 direction = mousePosition - transform.position;
6      float angle = Vector3.SignedAngle(Vector3.up, direction, Vector3.forward);
```

```
7       transform.rotation = Quaternion.Euler(0, 0, angle);
8   }
```

5.4 scripts/Spawner.cs, Enemy.cs y HealthEnemy.cs

5.4.1 Ubicación

`Assets/scripts`

Descripción general:
Generación, movimiento y vida de los enemigos.

5.4.2 Funciones

GenerateEnemy **Descripción:** Elige un ángulo al azar y crea el enemigo en ese punto de un círculo de radio 10,29 (justo fuera de la pantalla).

Código comentado:

```
1  private void GenerateEnemy()
2  {
3      spawnAngle = Random.Range(0, 2 * Mathf.PI);
4      Vector3 spawnDirection = new Vector3(Mathf.Cos(spawnAngle), Mathf.Sin(spawnAngle),
           0);
5      Vector3 spawnLocation = spawnDirection * 10.29f;
6      Instantiate(enemyTemplate, spawnLocation, Quaternion.identity);
7  }
```

Enemy.Update **Descripción:** Avanza hacia el jugador; si el jugador ha muerto, se aleja.

Código comentado:

```
1  moveDirection = (targetPosition.position - transform.position).normalized;
2  targetAttributes = target.GetComponent<AtributosJugador>();
3  if (targetAttributes.health > 0)
4      transform.position += moveDirection * moveSpeed * Time.deltaTime;
5  else
6      transform.position -= moveDirection * moveSpeed * Time.deltaTime;
```

Proyectil.OnCollisionEnter2D **Descripción:** Al tocar un enemigo le quita el daño multiplicado por la fuerza del jugador y desaparece.

Código comentado:

```
1  private void OnCollisionEnter2D(Collision2D collision)
2  {
3      if (collision.gameObject.CompareTag("Enemy"))
4      {
5          HealthEnemy enemyHealth = collision.gameObject.GetComponent<HealthEnemy>();
6          if (enemyHealth != null)
7          {
8              AtributosJugador playerAttributes = GameObject.FindGameObjectWithTag("
                   Player").GetComponent<AtributosJugador>();
9              enemyHealth.currentHealth -= damage * playerAttributes.strength;
10         }
11         Destroy(gameObject);
12     }
13 }
```

5.5 scripts/HealthPlayer.cs, GameEvents.cs y Score.cs

5.5.1 Ubicación

`Assets`

Descripción general:
La vida del jugador, los eventos de la partida y la puntuación.

5.5.2 Funciones

HealthPlayer.Update **Descripción:** Limita la vida al máximo, actualiza el color y, si llega a 0, lanza el evento de muerte y muestra la ventana con los puntos.

Código comentado:

```
if (playerAttributesScript.health > playerAttributesScript.maxHealth)
    playerAttributesScript.health = playerAttributesScript.maxHealth;

UpdateHealthColor();      // gradiente según health / maxHealth

if (playerAttributesScript.health <= 0)
{
    GameEvents.PlayerDied.Invoke();
    defeatScreen.SetActive(true);
    player.enabled = false;                    // desactiva las habilidades
    finalText.text = $"You got {score.score} points.";
}
```

GameEvents y Score **Descripción:** Eventos estáticos que cualquier script puede escuchar. El marcador suma un punto cuando muere un enemigo.

Código comentado:

```
public static UnityEvent PlayerDied = new UnityEvent();
public static UnityEvent EnemyDied = new UnityEvent();
public static UnityEvent GameStarted = new UnityEvent();

// Score
private void Start()
{
    GameEvents.EnemyDied.AddListener(EnemyDying);
}

public void EnemyDying()
{
    score++;
    UpdateScoreText();
}
```